

Neural networks, machine learning and randomness

Artificial neural networks based on randomness

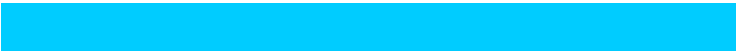
Optional course for master students

Martin Holeňa



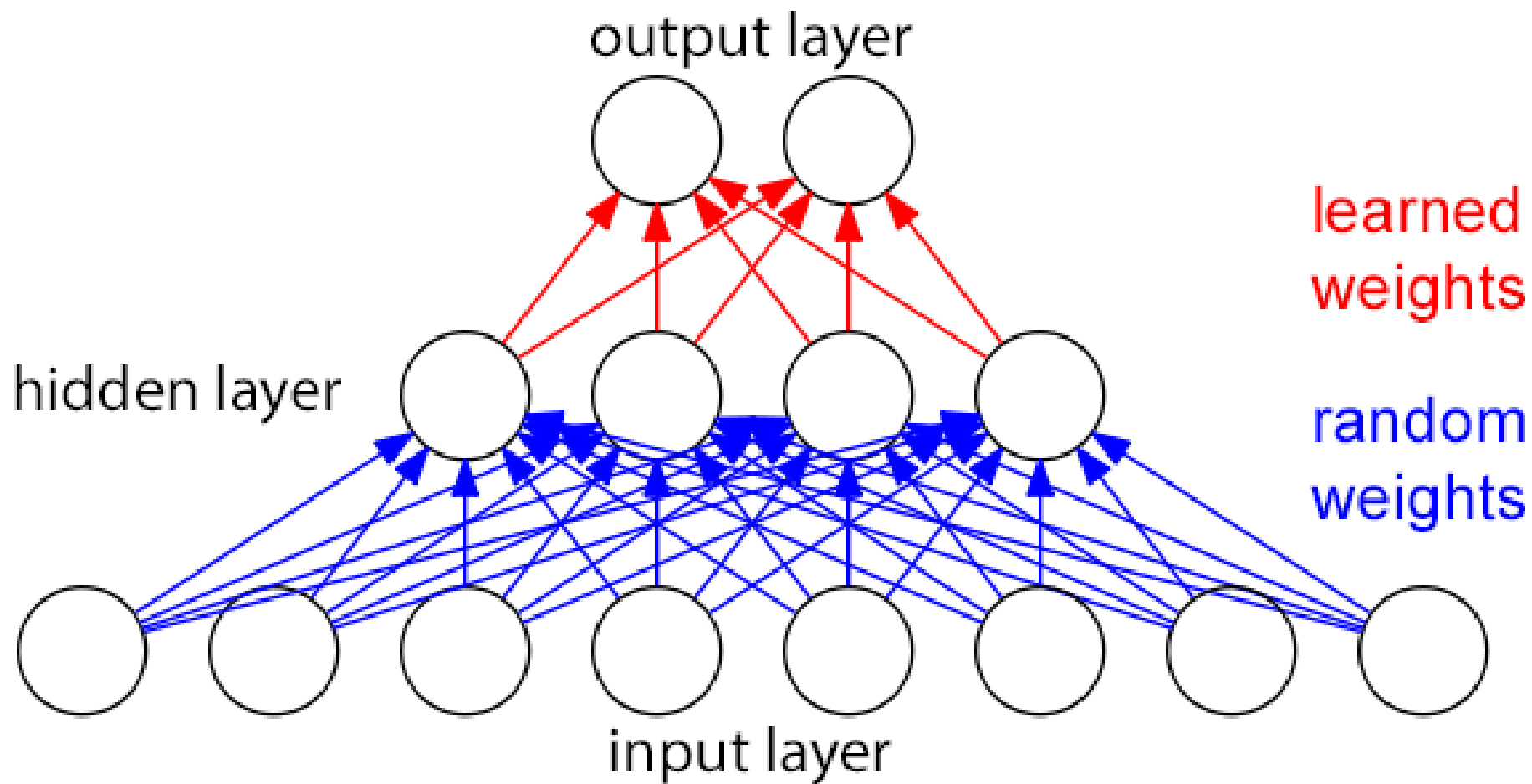


What will it be about?

- ◆ Extreme learning machines and their learning
 - ◆ Randomized variant of convolutional neural networks
 - ◆ Echo state machines and using inhibit connections
 - ◆ Bayesian kinds of artificial neural networks
 - Bayesian neural networks with random parameters
 - Bayesian neural networks with stochastic activation
- 
- 

Extreme learning machine (ELM)

- ◆ *Feedforward* ANN with *1 hidden layer*, scalar product outputs
 - layer sizes: input k , hidden l , output m neurons
- ◆ Activation function of the j^{th} hidden neuron: $a_j(\cdot | w_j, b_j): \mathbb{R}^k \rightarrow \mathbb{R}$
with the weight w_j and bias b_j , e.g. $a_j(\cdot | w_j, b_j) = \frac{1}{1 + e^{-(x^\top w_j + b_j)}}$
- ◆ $\Rightarrow$ ELM output for $x \in \mathbb{R}^k$ is $\sum_{j=1}^l \beta_j a_j(x | w_j, b_j)$, with $\beta_j \in \mathbb{R}^m$
- ◆ *Random*: w_j ($\sim$ synaptic operations), b_j (in $a_j \sim$ somatic operations)



Notation for ELM training data

◆ Input-target pairs $(x_1, t_1), \dots, (x_N, t_N) \in \mathbb{R}^k \times \mathbb{R}^m$

◆ Activities of the hidden neurons for $x \in \mathbb{R}^k$:

$$h(x) = (h_1(x), \dots, h_l(x)) = (a_1(x|w_1, b_1), \dots, a_l(x|w_l, b_l))$$

• allow to define a random kernel $K(x, y) = h(x)h(y)^\top$

◆ Matrix notation: $T = \begin{bmatrix} t_1^\top \\ \vdots \\ t_N^\top \end{bmatrix}, H = \begin{bmatrix} h(x_1) \\ \vdots \\ h(x_N) \end{bmatrix}$, thus H is random

ELM learning

- ◆ What is learnt? The *non-random* weights: $\beta = [\beta_1, \dots, \beta_l]^\top \in \mathbb{R}^{l \times m}$
- ◆ β is learnt through *minimizing* $\|\beta\|_1^{\sigma_1} + C\|H\beta - T\|_2^{\sigma_2}$
 - $\|\cdot\|_1, \|\cdot\|_2$ – matrix norms, $\sigma_1, \sigma_2 > 0, C \in (0, +\infty]$
 - $\|\beta\|_1^{\sigma_1}$ – regularization term, $\|H\beta - T\|_2^{\sigma_2}$ – error term
- ◆ If no regularization ($C = +\infty$), then $\arg \min \|H\beta - T\|_2^{\sigma_2} = H^+T$
 - H^+ – Moore-Penrose generalized inverse: $HH^+H = H, H^+HH^+ = H^+$

Optimization task for ELM learning

- ◆ Denote $\xi_{i,j} = t_{i,j} - h(x_i)\beta_j$, where $[t_{i,1}, \dots, t_{i,l}]^\top = t_i, i = 1, \dots, N$
- ◆ The ELM learning $\min \|\beta\|_1^{\sigma_1} + C \|H\beta - T\|_2^{\sigma_2}$ with $\sigma_1 = \sigma_2 = 2$ turns to constrained *optimization* $\min_{\beta} \sum_{j=1}^m \|\beta_j\|^2 + C \sum_{j=1}^m \sum_{i=1}^N \xi_{i,j}^2$ with *constraints* $t_{i,j} - h(x_i)^\top \beta_j - \xi_{i,j} = 0, i = 1, \dots, N, j = 1, \dots, m$
- ◆ Its Lagrangian L_{ELM} with Lagrange multipliers $\alpha_{i,j}$: $L = L_{\text{ELM}} = \sum_{j=1}^m \|\beta_j\|^2 + C \sum_{j=1}^m \sum_{i=1}^N \xi_{i,j}^2 + \sum_{j=1}^m \sum_{i=1}^N \alpha_{i,j} (t_{i,j} - h(x_i)\beta_j - \xi_{i,j})$

ELM optimization solution

- ◆ $\frac{\partial L}{\partial \beta_j} = 0 \Rightarrow \beta_j = \sum_{i=1}^N \alpha_{i,j} h(x_i)^\top$ for $i = 1, \dots, N$ and $j = 1, \dots, m$
 - *equivalently* in matrix notation: $\beta = H^\top \alpha$, where
 $\alpha = [\alpha_1, \dots, \alpha_N]^\top$ and $\alpha_i = [\alpha_{i,1}, \dots, \alpha_{i,m}]^\top$ for $i = 1, \dots, N$
- ◆ $\frac{\partial L}{\partial \xi_i} = 0 \Rightarrow \alpha_i = C \xi_i$, where $\xi_i = [\xi_{i,1}, \dots, \xi_{i,m}]^\top$ for $i = 1, \dots, N$
- ◆ $\frac{\partial L}{\partial \xi_i} = 0 \Rightarrow t_i^\top = h(x_i) \beta + \xi_i^\top$, i.e. the original constraints

ELM solution if $N > l$

- ◆ Combining $\beta = H^\top \alpha$ with $\alpha_i = C \xi_i$ yields $\beta = H^\top \xi \Rightarrow \xi = \frac{1}{C} (H^\top)^+ \beta$
- ◆ Considering also $t_i^\top = h(x_i) \beta + \xi_i^\top$ yields $H\beta - T + \frac{1}{C} (H^\top)^+ \beta = 0$
 $\Rightarrow H^\top \left(H + \frac{1}{C} (H^\top)^+ \right) \beta = H^\top T \Rightarrow$ the *learnt* $\beta = \left(\frac{I}{C} + H^\top H \right)^{-1} H^\top T$
- ◆ For the hidden neuron activities $h(x) = (h_1(x), \dots, h_l(x))$, the *ELM output* is the linear combination $h(x) \beta = h(x) \left(\frac{I}{C} + H^\top H \right)^{-1} H^\top T$

ELM solution if $N \leq l$

◆ Combining $\beta = H^\top \alpha$ with $\alpha_i = C \xi_i$ and $t_i^\top = h(x_i) \beta + \xi_i^\top$ yields

$$\left(\frac{I}{C} + HH^\top\right) \alpha = T, \beta = H^\top \left(\frac{I}{C} + HH^\top\right)^{-1} T \text{ and output } h(x) H^\top \left(\frac{I}{C} + HH^\top\right)^{-1} T$$

◆ With the kernel K , $K_{\text{ELM}} = \begin{bmatrix} K(x_1, x_1) & \cdots & K(x_1, x_N) \\ \vdots & \ddots & \vdots \\ K(x_N, x_1) & \cdots & K(x_N, x_N) \end{bmatrix} : \left(\frac{I}{C} + K_{\text{ELM}}\right) \alpha$

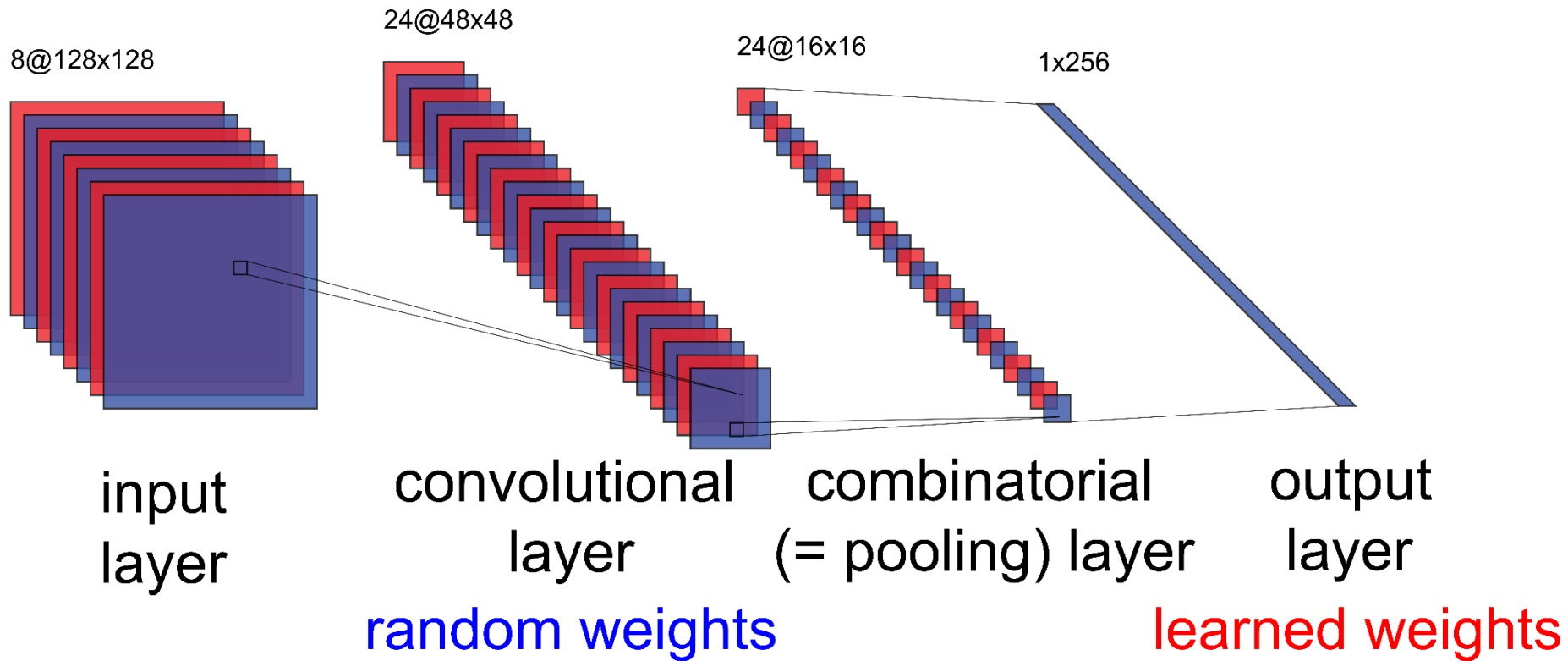
$$= T, \beta = H^\top \left(\frac{I}{C} + K_{\text{ELM}}\right)^{-1} T \text{ and output } \begin{bmatrix} K(x_1, x) \\ \vdots \\ K(x_N, x) \end{bmatrix}^\top \left(\frac{I}{C} + K_{\text{ELM}}\right)^{-1} T$$

ELM and random projection

- ◆ Random projection: Wx with $W \in \mathbb{R}^{r \times c}$ random (ELM: $W = \begin{bmatrix} w_1 \\ \vdots \\ w_l \end{bmatrix}$)
- ◆ Random projection differs from an ELM in 2 respects:
 1. It attempts to preserve Euclidean distance: $\|Wx - Wx'\| \cong \|x - x'\|$
 2. It has less rows than columns: $r < c$, whereas ELM usually has $l > k$, sometimes an order of magnitude larger

Randomized convolutional ANN

- ◆ Convolutional neural network (CNN) in which the weights from inputs to receptive fields (kernels)
i.e., *input layer* → *convolutional layer* are *random*
- ◆ Further supposed layers: combinatorial, fully connected
 - combinatorial performs pooling ⇒ has no weights
- ◆ Learned weights: combinatorial → output layer (fully connected)

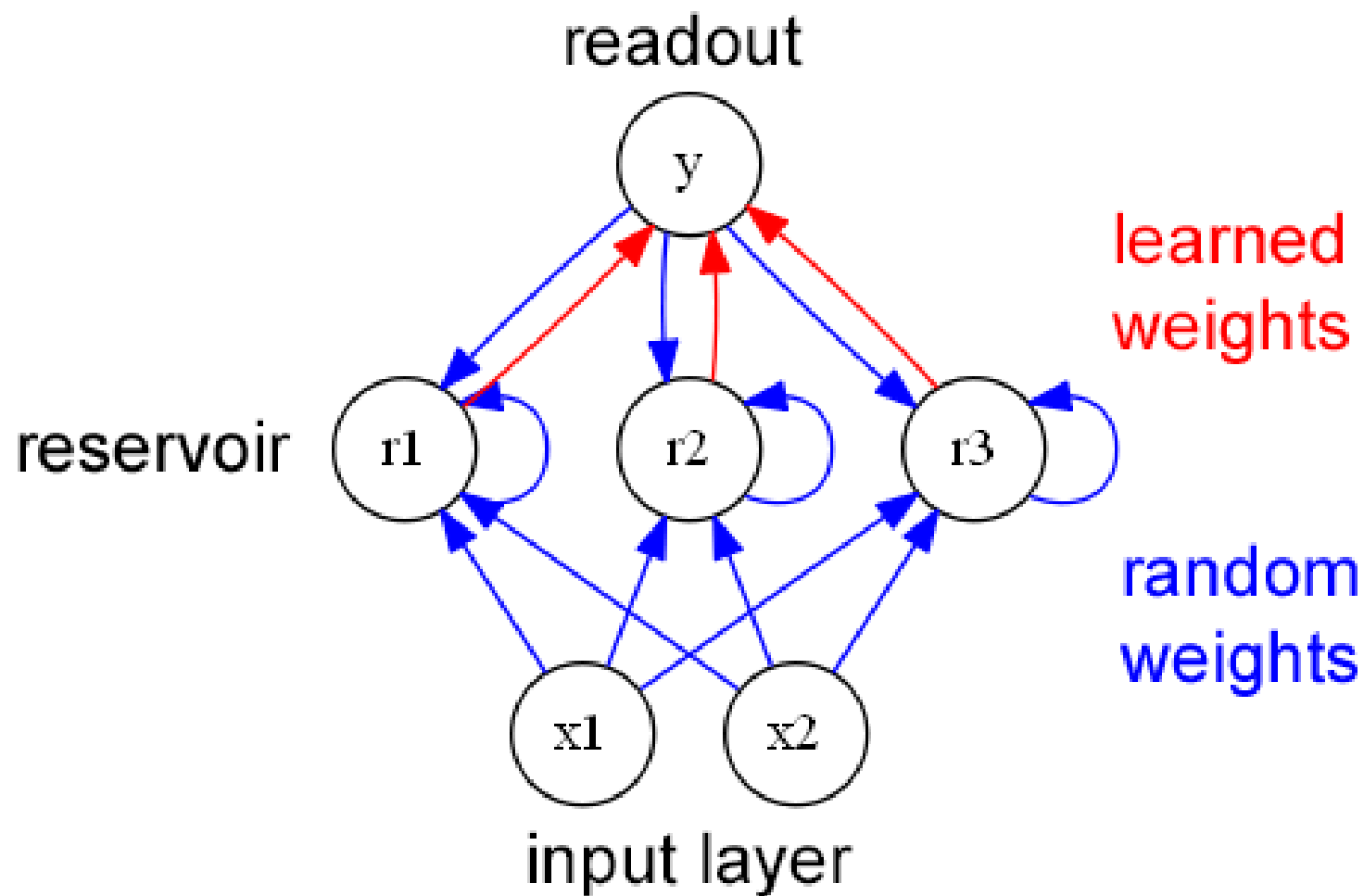


Properties of a randomized CNN

- ◆ If the receptive field size is $r \times r$ and the input dimension is d , then each convolutional layer map has the size $(d - r + 1) \times (d - r + 1)$
- ◆ The matrix A_m^{ic} of random weights between the input and convolutional layer is identical for any convolutional layer map m

Echo state network (ESN)

- ◆ *Recurrent* neural network with *random weights*
- ◆ Random are all weights to the hidden layer
 - connections from the input layer
 - + recurrent connection from itself + the output layer
- ◆ Weights hidden layer → output layer are learned
- ◆ ESN terminology: hidden layer – *reservoir*, output layer – *readout*



Activity evolution in an ESN

- ◆ Dimensions: input $x \in \mathbb{R}^d$, hidden layer $h \in \mathbb{R}^r$, output $y \in \mathbb{R}$
- ◆ Activity of the hidden layer for $t \in \mathbb{N}$:

$$h[t] = \alpha h[t-1] + (1 - \alpha) \sigma(W_{ir}x[t] + W_{rr}h[t-1] + w_{ro}y[t-1])$$

with $W_{ir} \in \mathbb{R}^{d \times r}$, $W_{rr} \in \mathbb{R}^{r \times r}$, $w_{ro} \in \mathbb{R}^r$, $\alpha \in \mathbb{R}$, σ – a nonlinearity

- if no momentum ($\alpha = 0$): $h[t] = \sigma(W_{ir}x[t] + W_{rr}h[t-1] + w_{ro}y[t-1])$

- ◆ Activity of the output: $y[t] = w_{io}^T x[t] + w_{ro}^T h[t]$, $w_{io} \in \mathbb{R}^d$, $w_{ro} \in \mathbb{R}^r$

ESN with inhibit connections

- ◆ In biological networks, *lateral inhibition* of an active neuron inhibit the response of other connected networks
- ◆ ESN simulate them with specific reservoir transformations:

$$W_{rr} = \begin{pmatrix} W_1 & -sI \\ -sI & W_2 \end{pmatrix}, s > 0, s \text{ is called } \textit{inhibition strength}$$

Stochastic configuration network (SCN)

- ◆ Network with d inputs, m outputs, 1 hidden layer of $L \geq 2$ neurons
- ◆ Used to approximate a function $f: D \subseteq \mathbb{R}^d \rightarrow \mathbb{R}$
- ◆ Computes an approximating function $f_L: D \rightarrow \mathbb{R}$, defined:

$$f_L(x) = \sum_{j=1}^L \beta_j g_j(w_j^\top x + b_j), g_j: \mathbb{R} \rightarrow \mathbb{R}$$

- $j = 1, \dots, L: g_j \in \Gamma$ — set of basis activation functions
- $j = 1, \dots, L: w_j$ — d -dimensional *random vector*, b_j — random *variable*

SCN for continuous functions

- ◆ f is assumed to be on D continuous, $f \in \mathcal{C}(D)$
- ◆ Basis activation functions are assumed with finite 2nd moments:

$$\forall g \in \Gamma: \int_{\mathbb{R}} |g(t)|^2 dt < \infty$$

- ◆ Then the f_L expectation w.r.t. the random w_j and b_j with increasing L approaches f in the following sense:

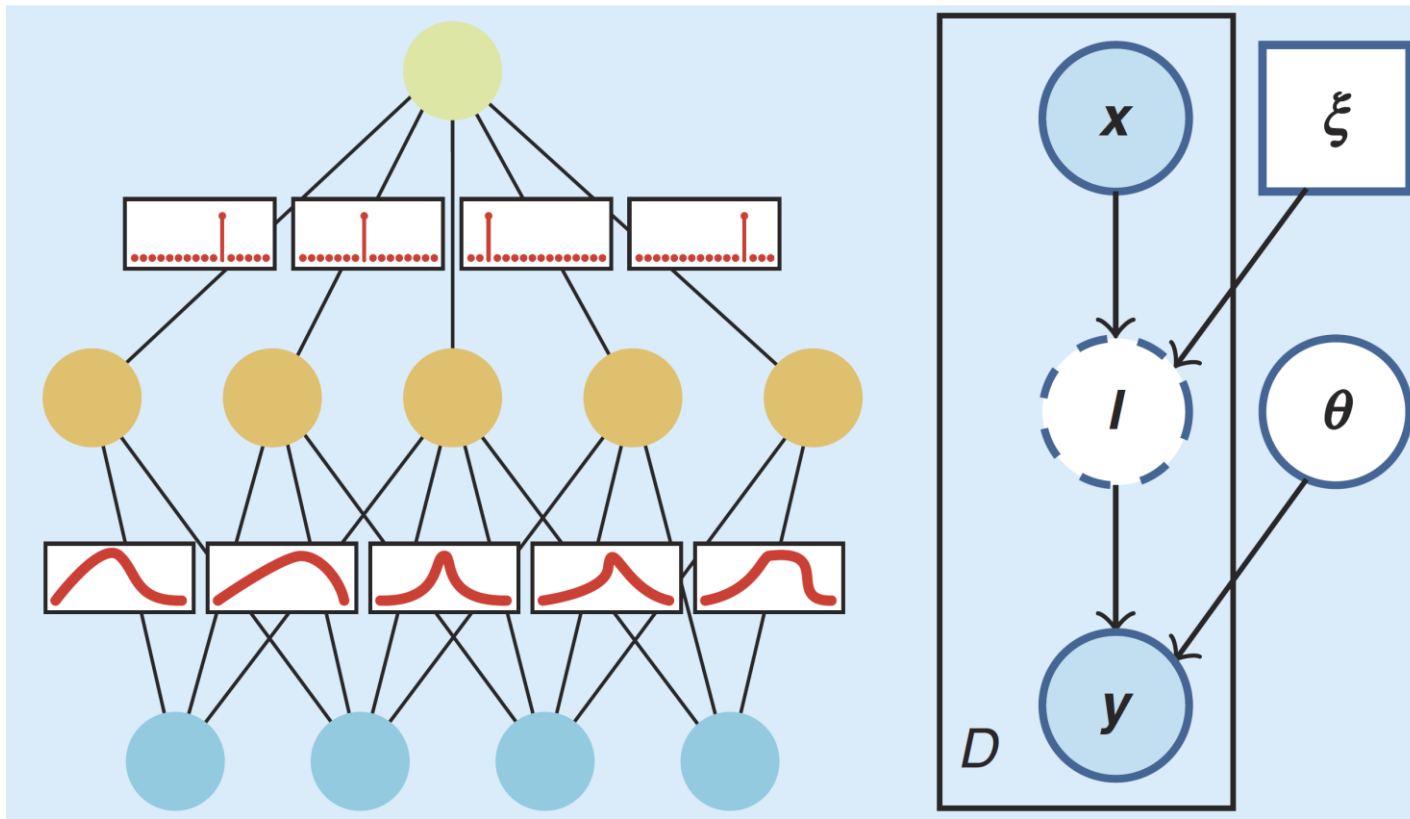
$$\lim_{L \rightarrow \infty} \mathbb{E} \left[\int_D |f(x) - f_L(x)|^2 dx \right] = 0$$

Bayesian neural network (BNN)

- ◆ Stochastic neural network trained *using* the *Bayesian approach*
- ◆ *Parameters* θ determining the function $y = F(x) = F_{\theta}(x)$ that the network learns, are viewed as *random variables*
 - prior distribution $p(\theta)$, posterior $p(\theta|D)$ conditioned on data D
 - data are $D = \{(x_1, y_1), \dots, (x_p, y_p)\}$, denote $D_x = \{x_1, \dots, x_p\}$, $D_y = \{y_1, \dots, y_p\}$
- ◆ Often with superposed random noise ϵ : $y = F(x) + \epsilon$

BNNs with restricted stochasticity

- ◆ Only parameters of 1/several last layer(s) are random
- ◆ Suitable representation for them: probabilistic graphical model



Computing a BNN prediction

- ◆ BNN prediction: a *random variable* with a *distribution* $p(y|x, D)$
 - a stochastic approximation of $F(x)$ for an input x
 - computed using the posterior $p(\theta|D)$: $p(y|x, D) = \int p(y|x, \theta') p(\theta'|D) d\theta'$
- ◆ Provided the inputs D are independent of model parameters θ ,
the posterior fulfills the Bayes theorem $p(\theta|D) = \frac{p(D_y|D_x, \theta) p(\theta)}{\int p(D_y|D_x, \theta') p(\theta') d\theta'}$

BNN distributional assumptions

- ◆ The distribution of θ is usually assumed Gaussian: $\theta \sim N(\mu, \Sigma)$
- ◆ For BNNs performing *regression*, the predictive distribution of y $p(y|x, D)$ is assumed Gaussian with same variance: $y \sim N(F_\theta, \Sigma)$
- ◆ For BNNs performing *classification*, $p(y|x, D)$ is categorical with the set of categories given by $F_\theta(x)$: $y \sim \text{Cat}(F_\theta(x))$
- ◆ In any case, for the whole dataset $p(D_y | D_x, \theta) = \prod_{(x,y) \in D} p(y|x, \theta)$

BNN estimate of the output

- ◆ An estimate $\hat{y}$ of y relies on sampling θ from data D
 - a set Θ is sampled from the distribution of θ
- ◆ If the network performs *regression*: $\hat{y}(x) = \frac{1}{|\Theta|} \sum_{\theta \in \Theta} F_{\theta}(x)$
 - it has the covariance $\text{cov}(\hat{y}|x, D) = \frac{1}{|\Theta|-1} \sum_{\theta \in \Theta} (F_{\theta}(x) - \hat{y})(F_{\theta}(x) - \hat{y})^T$
- ◆ If it *classifies* into classes $c = 1, \dots, C$: $\hat{y}(x) = \arg \max_c \hat{p}_c$
 - $\hat{p}_c$ is the estimated probability of c : $\hat{p}_c = \frac{1}{|\Theta|} |\{\theta \in \Theta | F_{\theta}(x) = c\}|$

BNNs with stochastic activation

- ◆ Random are not parameters, but activation function inputs
 - their distributions depend on outputs from previous layers
- ◆ For a BNN with *layers* $L_0, \dots, L_n$, *activation function* a :

$L_0(x) = x$, inter-layer step $L_k(x) = a(\theta_k(x))$, and $L_n(x) = y$

- random is $\theta_k(x) \sim N(W_k L_{k-1}(x) + b_k, \Sigma)$ with W_k – matrix, b_k – vector

$$p(D_y, L_1(x), \dots, L_{n-1}(x) | D_x) = \prod_{(x,y) \in D} \prod_{k=1}^n p(L_k(x) | L_{k-1}(x))$$

Activation $\bowtie$ parameter stochasticity

- ◆ Consider a BNN with layers $L_0, \dots, L_n$, activation function a and a step $L_k(x) = a(WL_{k-1}(x) + b)$ with $W \sim N(\mu_W, \Sigma_W), b \sim N(\mu_b, \Sigma_b)$
- ◆ It can be shown *equivalent to* stochastic activation

$$L_k(x) = a(\theta(x)), \theta(x) \sim N(\mu_W L_{k-1}(x) + \mu_b, (\otimes_{k-1})^T \Sigma_W \otimes_{k-1} + \Sigma_b)$$

$$\text{where } \otimes_{k-1} = \begin{pmatrix} L_{k-1}(x) & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & L_{k-1}(x) \end{pmatrix}$$

Setting BNN priors

- ◆ Choosing the parameter prior $p(\theta)$ – often not intuitive
- ◆ *Frequently* used default is uncorrelated normal *prior*: $\theta \sim N(\sigma I)$
 - however, it is not supported by theoretical arguments
- ◆ Connection of priors with BNN learning:
 - parameter learning from the loss yields $\hat{\theta} = \arg \max_{\theta} p(D_y | D_x, \theta)$
 - Using the prior $\Rightarrow$ *posterior learning* $\hat{\theta} = \arg \max_{\theta} p(D_y | D_x, \theta) p(\theta)$

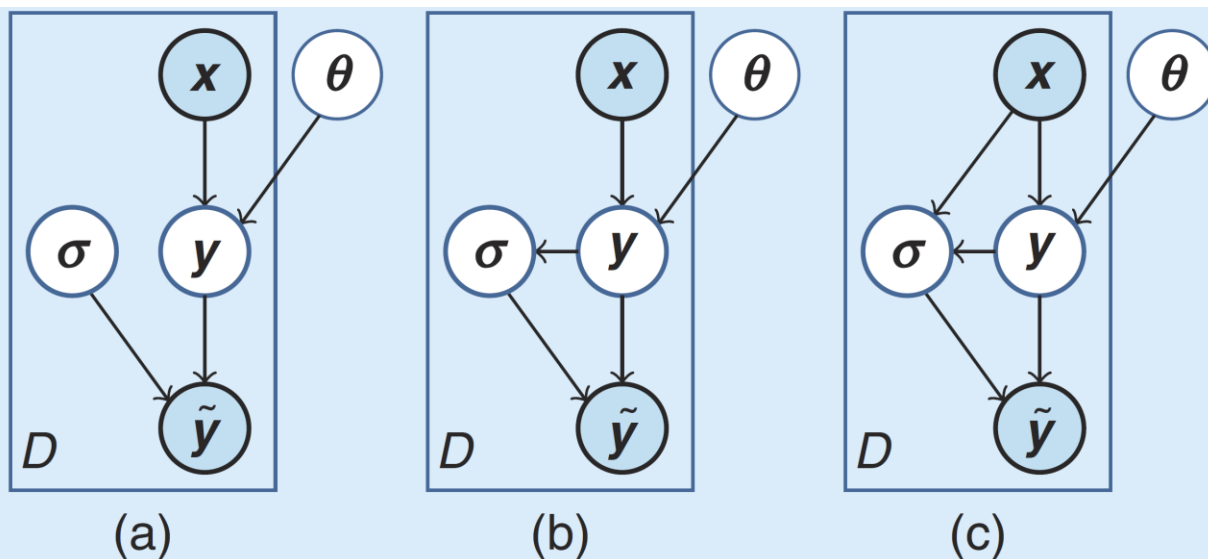
Noise in BNNs

◆ *3 noise models*: noise completely at random (a),

noise at random (b), noise not at random (c)

- they can be represented as *probabilistic graphical models*:

$y + \text{noise} = \hat{y}$, dependences are represented with directed edges



Data augmentation for BNNs

◆ Data augmentation in general complements the collected data with results of transforming them with a transformation entailing no or only predictable label change.

◆ For a BNN, it *entails changing the posterior*:

$$p(\theta|D) \rightarrow p(\theta|D, \text{Augment}) \propto p(\theta) \int p(y|x', \theta) p(x'|x, \text{Augment}) dx'$$

- Augment = distribution of the augmentation results

BNNs and back-propagation

- ◆ For a BNN loss function L , back-propagating $\nabla_{\theta} L$ is not possible due to the stochasticity of θ
- ◆ Getting around this problem is called *reparametrization trick*:
 $\theta = t(\varepsilon, \phi), \varepsilon \sim Q$, with a parameter $\phi \in \mathbb{R}$ and a fixed Q
 - the non-stochasticity of ϕ allows *back-propagating* $\nabla_{\phi} L$

Hierarchical BNNs

- ◆ Several *parameters* $\theta_1, \dots, \theta_I$ *depend on* another common *parameter* $\xi \Rightarrow$ the joint *probability* of $\theta_1, \dots, \theta_I, \xi$ is

$$p(\theta_1, \dots, \theta_I, \xi | D_1, \dots, D_I) \propto p(\xi) \prod_{i=1}^I p(\theta_i | \xi) p(D_{i,y} | D_{i,x}, \theta_i)$$

- ◆ Can be used for *metalearning* of BNNs:
 - the parameters $\theta_1, \dots, \theta_I$ correspond to features of I BNNs
 - the parameter ξ corresponds to their common metafeatures

Advantages of BNNs

1. They are a natural approach to *quantify uncertainty*.
2. Points out of the training distribution are predicted with high $p(\theta|D)$ (called high *epistemic* uncertainty)
 - instead of blindly giving a wrong prediction
 - allows *inference*: draw $\theta_i \sim p(\theta|D)$ and infer $y_i = F_{\theta_i}(x), i = 1, \dots, N$
3. The prior distribution of θ is made explicit

BNNs in active learning

- ◆ Based on estimating the *uncertainty of $\hat{y}(x)$* :
 1. A set of samples is drawn, defined $\Theta = \{\theta_i | i = 1, \dots, N, \theta_i \sim p(\theta | D)\}$
 2. The uncertainty is *estimated with* $\Sigma_x = \frac{\sum_{\theta \in \Theta} (F_{\theta}(x) - \hat{y}(x))(F_{\theta}(x) - \hat{y}(x))^{\top}}{|\Theta| - 1}$
- ◆ Among the unevaluated points x available for evaluation is evaluated the one maximizing the uncertainty Σ_x
 - evaluation – regression: obtaining the value, classification: labelling